Project Proposal: Concurrent, Low-Latency Order Matching Engine

Course: Operating Systems **Team size:** 5 **Duration:** ~8 weeks

1. What We're Building

A **matching engine** — the kind of system that sits at the core of any stock exchange or trading platform. It takes incoming buy/sell orders, matches compatible ones against each other, and produces trades. On the surface this sounds like a finance problem, but underneath it's a textbook operating-systems problem: many things happening at once, all fighting for CPU time, memory, and I/O, with strict timing requirements.

We're building this specifically because it forces us to *use* every major OS concept from this semester for a real reason, rather than as an isolated toy exercise — and because the entire project is centered on one theme: **measuring and reducing latency**, which is what makes it a genuine systems project rather than a checklist of unrelated demos.

2. Background: The Two Concepts at the Heart of This

Order book — a data structure holding all currently active buy and sell orders, organized by price. When a new order arrives, the engine checks whether it can be matched against an existing order on the opposite side (a buyer willing to pay ≥ what a seller is asking). If yes, a trade happens; if no, the order is added to the book and waits.

Matching engine — the component that owns the order book and processes incoming orders against it, one at a time, following price-time priority rules (best price first, earliest order first among equal prices).

That's the whole "business logic." The interesting engineering problem is everything around it: how do multiple producers feed orders into this engine without blocking each other, how is memory managed so allocation doesn't cause unpredictable delays, and how is the OS scheduler configured so this thread isn't randomly paused mid-work.

---

3. Components (and the OS Concept Each One Demonstrates)

Component	What it does	OS Concept Covered
Matching engine core	Single-threaded, correctness-first implementation of the order book and matching logic.	Baseline / data structures
Order ingestion queue	A **ring buffer** — a fixed-size circular queue that passes orders from producer threads to the matching engine. Built two ways: (a) protected by a mutex, (b) **lock-free**, using atomic CPU instructions instead of locks so threads never block each other while enqueuing/dequeuing.	Synchronization & concurrency
Custom memory allocator	A pool/slab allocator that pre-reserves memory for order objects instead of calling `malloc` / `free` per order. General-purpose allocators have unpredictable timing under load; a purpose-built allocator avoids that.	Memory management
Scheduling experiments	Pin the matching thread to a specific CPU core (`sched_setaffinity`) and test real-time scheduling policies (`SCHED_FIFO` / `SCHED_RR`) against the Linux default (`SCHED_OTHER`), measuring how each affects timing consistency.	CPU scheduling
Network ingestion layer	Accepts orders over a socket, implemented and compared two ways: traditional `epoll`	I/O systems

Component	What it does	OS Concept Covered
	vs. the newer `io_uring` interface.	
Benchmarking harness	Generates load, records how long each order takes to process, and produces latency distributions (not just averages — this is where the real findings come from).	Measurement / evaluation
(Stretch goal) **Write-ahead log**	Persists every order to disk before processing, so the system can recover its state after a crash.	File systems

---

4. Why We're Comparing, Not Just Building

Every core component above is built in **two versions** on purpose (mutex vs. lock-free queue, default vs. real-time scheduling, `epoll` vs. `io_uring`). The deliverable isn't "it works" — it's a measured, explained answer to *why one approach is faster or more consistent than the other, and under what conditions*. That comparison is the actual OS learning outcome; the trading-system framing is just what makes the comparison concrete and interesting to build.

---

5. Team Roles

Member	Owns
A	Matching engine core & order book correctness
B	Ring buffer — mutex-based and lock-free versions
C	Custom memory allocator
D	Scheduling experiments & CPU affinity
E	Network ingestion (epoll/io_uring) & benchmarking harness

Each role maps to one syllabus unit, built against a shared, stable core, so every member can independently explain and defend their piece.

6. Timeline (High Level)

Weeks	Focus
1–2	Single-threaded core: correct, tested order book & matching logic
3–5	Parallel component development (queue, allocator, scheduling, I/O) against the stable core
6–7	Integration
8	Benchmarking, analysis, write-up

7. Deliverables

- Working C/C++ source code (all versions: baseline and optimized)
- Benchmark report with latency measurements (p50/p99/p999, not just averages) and an explanation of *why* the numbers look the way they do
- Short write-up connecting each component back to the OS concept it demonstrates

8. Tools

Modern C++ (C++17/20), Linux, `std::thread` / `std::atomic` , `epoll` , `io_uring` , `perf` , ThreadSanitizer/AddressSanitizer for correctness verification.

Appendix: Technical Detail (for anyone who wants to go deeper)

Order book algorithm (price-time priority): incoming buy orders match against the lowest-priced resting sell if `buy.price >= sell.price` , filling by best price first and, within a price level, earliest order first. Unmatched quantity rests in the book at its price level. Implemented with two `std::map` s (one per side, ordered by price) each holding a FIFO queue of orders at that price.

Ring buffer, mutex version: a fixed-size circular array with `push` / `pop` protected by a single `std::mutex` — the correctness baseline.

Ring buffer, lock-free version: single-producer/single-consumer, using two `std::atomic<size_t>` indices (head, tail) with `memory_order_acquire` / `memory_order_release` instead of a mutex — the producer and consumer never block each other, at the cost of significantly more careful reasoning about memory visibility.

Slab allocator: one large pre-reserved memory block, carved into fixed-size chunks matching the `Order` object size, managed via an in-place free list (no separate bookkeeping memory) — avoids `malloc` 's unpredictable per-call timing in the hot path.

Scheduling: the matching thread is pinned to a dedicated CPU core (`sched_setaffinity`) and tested under both the default scheduler (`SCHED_OTHER`) and a real-time policy (`SCHED_FIFO`), measuring the effect on tail latency (p99/p999) specifically, since that's where scheduler-induced jitter shows up — not in the average.

Network layer: two implementations of the same interface — `epoll` (readiness-based; one syscall per read-ready notification) and `io_uring` (kernel/user-space shared ring buffers, batching submissions and completions to cut syscall overhead) — compared under identical load.

Measurement methodology: every order is timestamped at creation; the benchmarking harness records the full latency from creation to trade generation for every order across a test run, and reports p50/p99/p999 (not just the mean), since tail behavior is what actually differentiates the design choices being compared.